COMPTE RENDU DE TP

IoT & Énergies renouvelables

Étude de la carte ESP32 TTGO LoRa32 avec écran OLED — explication du programme ligne par ligne

Réalisé par : **Yousri Elouej**, Anis Boubakri

Classe : LISI 3

1. Objectif du TP

L’objectif est d’utiliser une carte ESP32, un écran OLED et les bibliothèques Arduino pour afficher des informations sur le microcontrôleur. Le programme initialise la communication série, le bus I²C et l’écran, configure le Wi-Fi en mode station, puis affiche des informations système et l’adresse MAC.

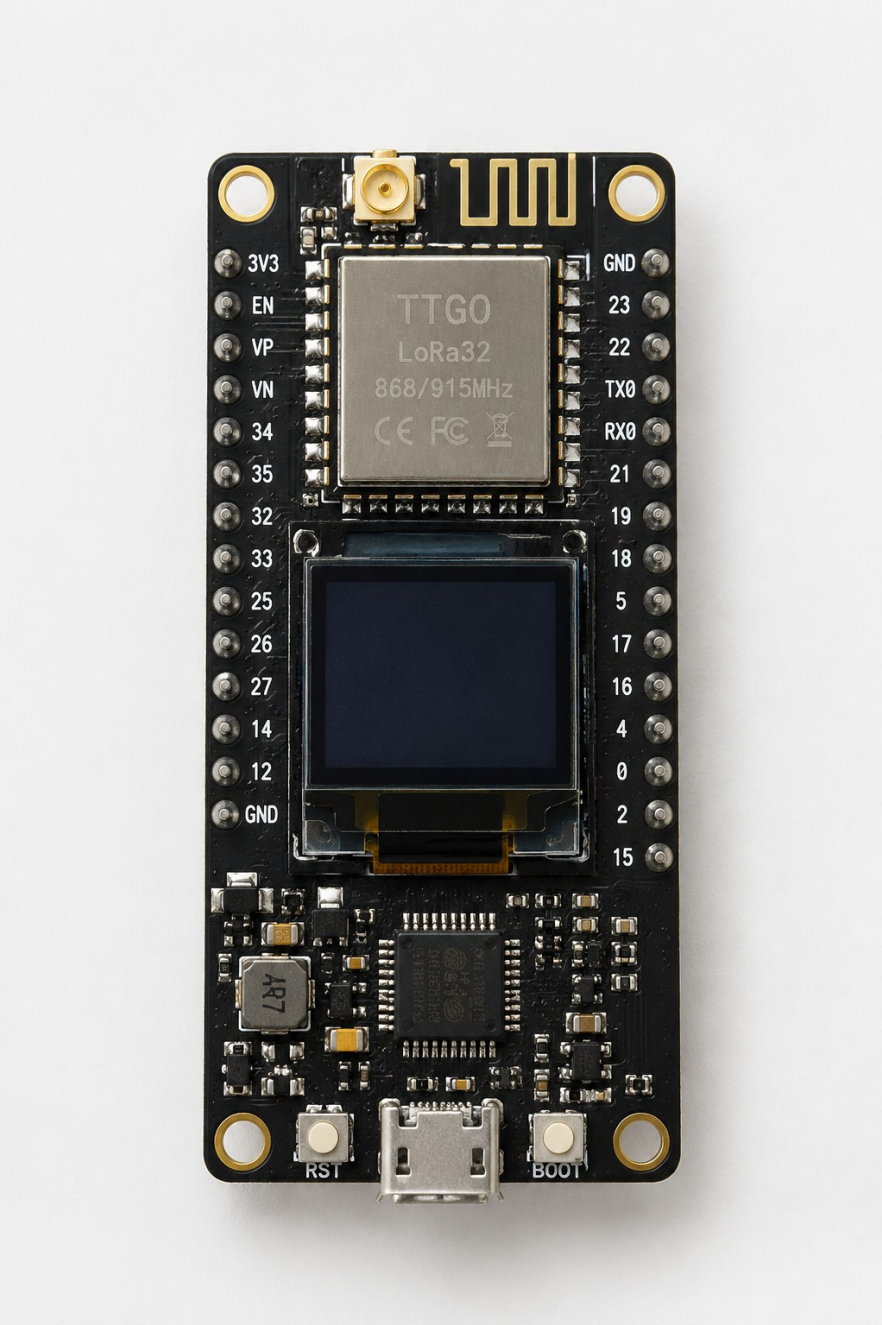


Figure 1 — Carte ESP32 TTGO LoRa32 avec écran OLED intégré.

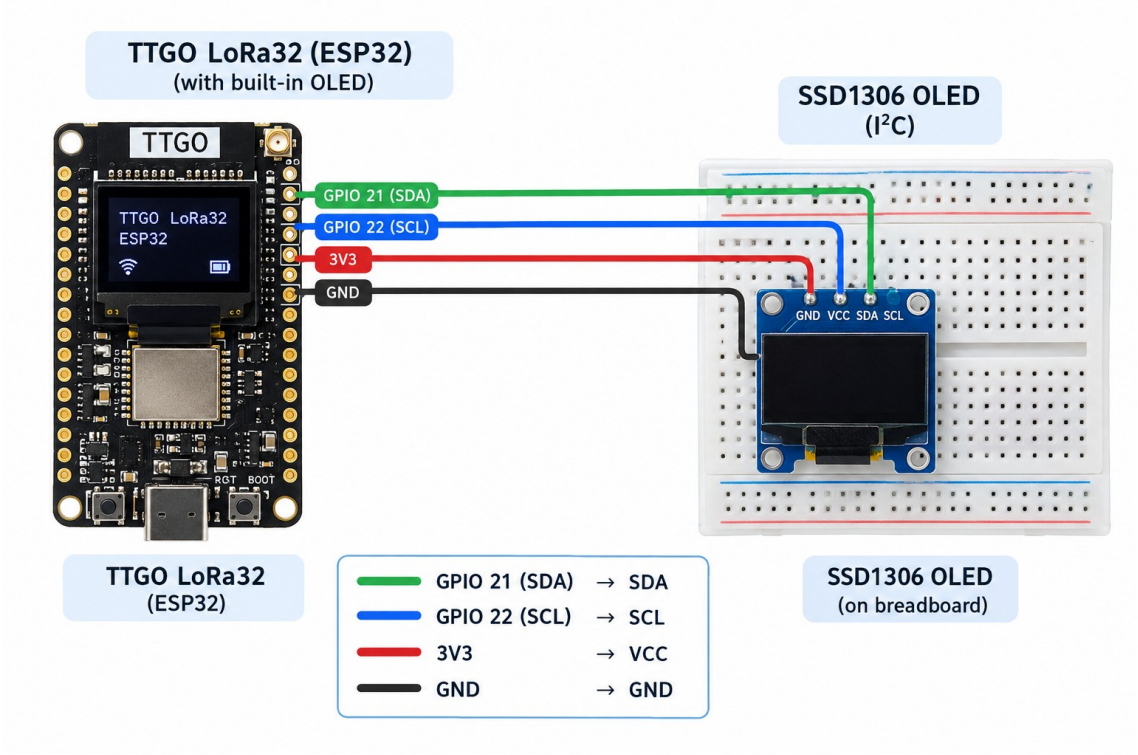


Figure 2 — Schéma illustratif d'une liaison I²C avec écran OLED SSD1306 externe.

Attention : ce schéma montre un écran OLED externe à titre pédagogique ; sur la TTGO LoRa32 utilisée ici, l'écran est déjà intégré à la carte. Les broches et le câblage doivent être vérifiés selon le modèle exact.

2. Code visible sur les photos

Le code ci-dessous reprend les instructions que l'on peut lire sur les photos. Quelques caractères de la partie centrale étant flous, il faut comparer les lignes printf avec le PDF original avant de les considérer comme une transcription exacte.

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 oled(128, 64, &Wire, -1);

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.setTextColor(SSD1306_WHITE);
  WiFi.mode(WIFI_STA);
  delay(500);

  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setCursor(0, 0);
  oled.println("--- Infos ESP32 ---");
  oled.printf("Modèle: %s\n", ESP.getChipModel());
  oled.printf("Coeurs: %d | %d MHz\n", ESP.getChipCores(), ESP.getCpuFreqMHz());
```

```
  oled.printf("Flash : %d Mo\n", ESP.getFlashChipSize() / (1024 * 1024));
  oled.printf("RAM : %d o\n", ESP.getFreeHeap());
  oled.println("MAC:");
  oled.println(WiFi.macAddress());
  oled.display();
  Serial.println("MAC : " + WiFi.macAddress());
}

void loop() { }
```

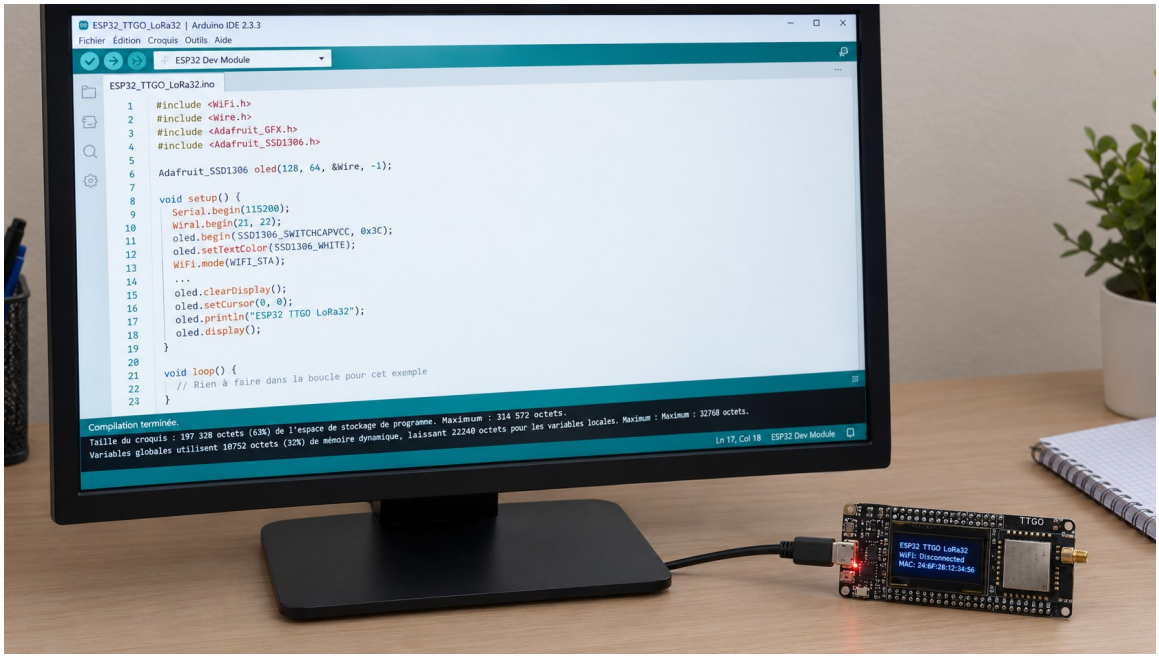


Figure 3 — Programmation de l'ESP32 dans l'environnement Arduino IDE.

L'IDE Arduino sert à écrire le programme, sélectionner la carte et le port, compiler le code puis le téléverser vers l'ESP32. L'image est illustrative : le code et les messages affichés peuvent différer du TP réel.

3. Explication ligne par ligne

#include <WiFi.h> Inclut la bibliothèque Wi-Fi de l'ESP32. Elle fournit les fonctions pour configurer le Wi-Fi et lire l'adresse MAC de l'interface réseau.

#include <Wire.h> Inclut la bibliothèque Wire qui permet de communiquer avec des périphériques utilisant le bus I²C.

#include <Adafruit_GFX.h> Ajoute les fonctions graphiques générales : texte, lignes, formes et gestion de l'affichage.

#include <Adafruit_SSD1306.h> Ajoute le pilote adapté aux écrans OLED utilisant le contrôleur SSD1306.

Adafruit_SSD1306 oled(128, 64, &Wire, -1); Crée l'objet oled. L'écran a 128 pixels de largeur et 64 pixels de hauteur. &Wire indique l'utilisation du bus I²C. -1 signifie qu'aucune broche RESET dédiée n'est indiquée.

void setup() { Déclare setup(), une fonction exécutée une seule fois au démarrage ou après la réinitialisation.

Serial.begin(115200); Démarre la communication série à 115200 bauds. Le moniteur série doit être réglé sur la même vitesse.

Wire.begin(21, 22); Initialise I²C avec GPIO 21 comme SDA (données) et GPIO 22 comme SCL (horloge). Ces broches doivent correspondre au modèle de carte utilisé.

oled.begin(SSD1306_SWITCHCAPVCC, 0x3C); Initialise l'écran OLED. SSD1306_SWITCHCAPVCC sélectionne le mode d'alimentation interne prévu par la bibliothèque. 0x3C est l'adresse I²C de l'écran ; certains modules peuvent utiliser une autre adresse.

oled.setTextColor(SSD1306_WHITE); Choisit la couleur blanche pour le texte. Sur un écran monochrome, les pixels du texte sont allumés.

WiFi.mode(WIFI_STA); Place le Wi-Fi en mode station : l'ESP32 peut se connecter à un routeur ou point d'accès. Cette ligne ne se connecte pas à un réseau à elle seule.

delay(500); Suspend le programme pendant 500 millisecondes, soit une demi-seconde.

oled.clearDisplay(); Efface le tampon graphique en mémoire. Pour que l'effacement apparaisse sur l'écran physique, il faut ensuite appeler oled.display().

oled.setTextSize(1); Choisit la taille standard du texte.

oled.setCursor(0, 0); Place le curseur au pixel (0, 0), en haut à gauche de l'écran.

oled.println("--- Infos ESP32 ---"); Écrit un titre sur l'écran puis passe à la ligne suivante.

oled.printf("Modèle: %s\\n", ESP.getChipModel()); Affiche le modèle du microcontrôleur. %s est un emplacement de texte remplacé par la valeur retournée par ESP.getChipModel(). \\n représente le saut de ligne dans la chaîne C/C++.

oled.printf("Coeurs: %d | %d MHz\\n", ESP.getChipCores(), ESP.getCpuFreqMHz()); %d affiche un nombre entier. ESP.getChipCores() renvoie le nombre de cœurs du processeur et ESP.getCpuFreqMHz() renvoie sa fréquence en MHz. Vérifier la formulation exacte de cette ligne dans le PDF original.

oled.printf("Flash : %d Mo\\n", ESP.getFlashChipSize() / (1024 * 1024)); Affiche la taille de la mémoire Flash en mégaoctets. ESP.getFlashChipSize() renvoie une taille en octets ; la division par 1024 × 1024 convertit en MiB, souvent notés Mo dans les supports de TP.

oled.printf("RAM : %d o\\n", ESP.getFreeHeap()); Affiche la quantité de mémoire heap libre en octets. Cette valeur est la mémoire dynamique disponible, et non la totalité de la RAM physique.

oled.println("MAC:"); Affiche l'étiquette MAC sur l'écran et passe à la ligne.

oled.println(WiFi.macAddress()); Affiche l'adresse MAC de l'interface Wi-Fi de l'ESP32. Une adresse MAC identifie l'interface réseau sur le réseau local.

oled.display(); Transfère le contenu du tampon graphique vers l'écran OLED. Sans cet appel, les textes écrits dans le tampon peuvent ne pas apparaître physiquement.

Serial.println("MAC : " + WiFi.macAddress()); Envoie également l'adresse MAC au moniteur série, suivie d'un retour à la ligne.

} Ferme la fonction setup().

void loop() { } Déclare la fonction loop(), exécutée continuellement par le framework Arduino. Comme elle est vide ici, aucune opération supplémentaire n'est répétée après l'exécution de setup().

4. Résumé du déroulement

1. Les bibliothèques nécessaires sont chargées.
2. L'ESP32 démarre la liaison série et initialise le bus I²C.
3. L'écran OLED est initialisé et son style d'affichage est défini.
4. Le Wi-Fi est configuré en mode station.
5. Le tampon de l'écran est effacé, puis les informations système sont écrites.
6. oled.display() envoie le contenu à l'écran ; l'adresse MAC est aussi envoyée au moniteur série.
7. La fonction loop() est vide : l'affichage est donc effectué une seule fois, au démarrage.

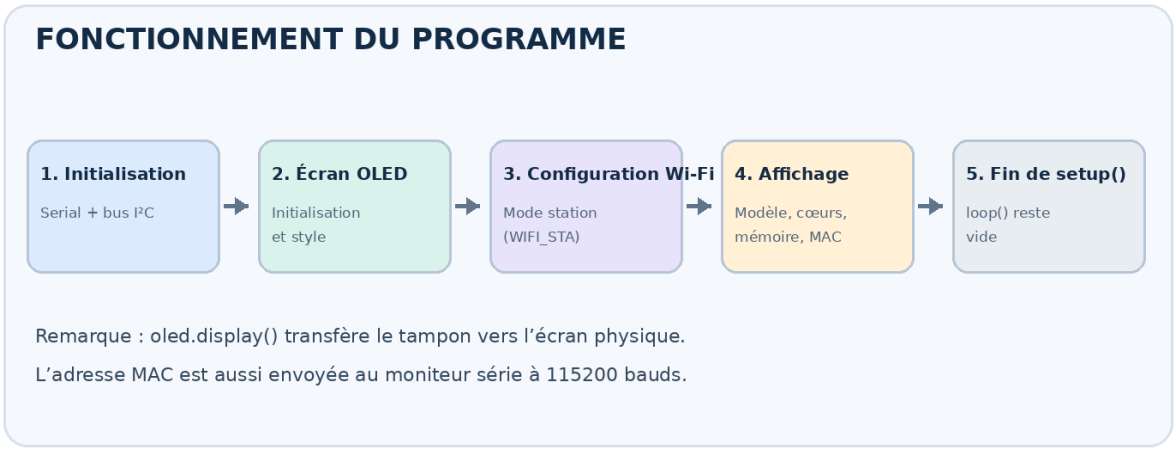


Figure 2 — Étapes principales exécutées par le programme.

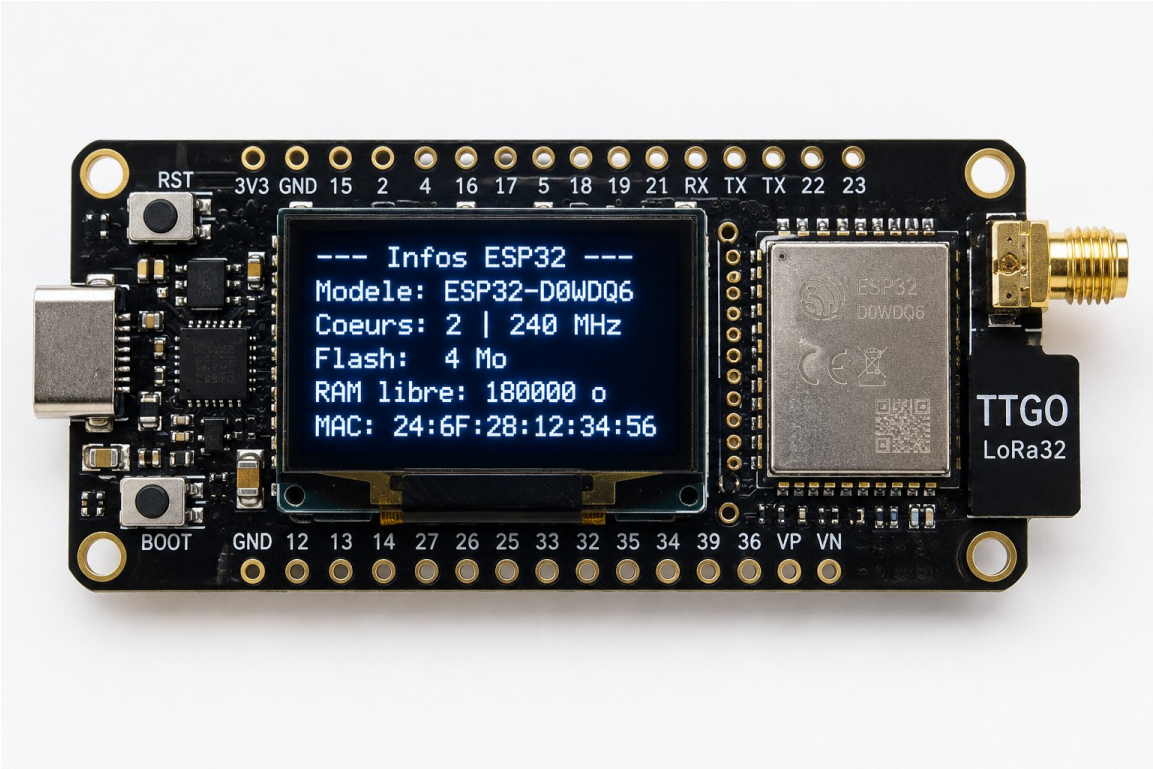


Figure 4 — Exemple illustratif du résultat affiché sur l'écran OLED.

Le programme affiche les informations système de l'ESP32 et son adresse MAC. Les valeurs visibles sur cette illustration sont fictives et peuvent différer de celles obtenues lors de l'exécution réelle.

5. Problèmes possibles et solutions

Problème possible	Solution à vérifier
Écran noir ou vide	Vérifier l'alimentation, SDA/SCL, l'adresse I²C 0x3C et le résultat de oled.begin().
Bibliothèque introuvable	Installer Adafruit GFX Library et Adafruit SSD1306 dans le gestionnaire de bibliothèques Arduino.
Moniteur série vide	Sélectionner le bon port et régler le débit sur 115200 bauds.
Échec de téléversement	Vérifier la carte sélectionnée, le port, le câble USB et, si nécessaire, la procédure BOOT de la carte.

Valeurs système différentes	Les valeurs dépendent du modèle de l'ESP32 et de la configuration de la carte.

6. Lien avec l'IoT et les énergies renouvelables

L'ESP32 peut servir de base à un objet connecté : il peut lire des capteurs, afficher localement les mesures et transmettre les données par Wi-Fi. Pour une application d'énergie renouvelable, on pourrait ajouter des capteurs de tension, de courant ou de température afin de suivre le fonctionnement d'un panneau solaire. Le programme visible ici affiche surtout des informations système ; il ne mesure pas directement l'énergie produite.

7. Conclusion

Ce programme montre comment initialiser un écran OLED, utiliser le bus I²C, configurer le Wi-Fi et afficher des informations du microcontrôleur ESP32. L'explication ligne par ligne permet de comprendre le rôle de chaque bibliothèque et de chaque fonction. Ces éléments peuvent ensuite être réutilisés dans un projet IoT de suivi d'une installation à énergie renouvelable.

Remarque

Cette explication est fondée sur les lignes lisibles dans les deux photos fournies. Pour expliquer exactement l'ensemble du code du professeur, y compris les lignes non visibles ou floues, il faut joindre le PDF complet ou des captures nettes des pages contenant le programme.